

UNIVERSIDAD TECNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computacion y Diseno Digital

Carrera de Ingenieria de Software

Taller GA-U3 - Diseno de arquitectura segun atributos de calidad

Escenario asignado: Plataforma de streaming de video sobre demanda

Asignatura: Aplicaciones Web - 5to nivel

Periodo academico: 2026-2027 PPA

Quevedo - Los Rios - Ecuador - 2026

1. Contexto del escenario

El escenario analizado corresponde a una plataforma de streaming de video sobre demanda operada por una compania de alcance nacional, que compite con proveedores globales y factura mediante suscripcion mensual e ingresos publicitarios. El consumo de video representa una fraccion creciente del trafico de red a nivel mundial, y la infraestructura de entrega -almacenamiento, transcodificacion y distribucion- es hoy uno de los factores que mas condiciona la experiencia percibida por el usuario final[1].

Los actores del sistema son el suscriptor final, que accede desde aplicaciones moviles, navegador web, Smart TV y consolas; el editor de contenido, encargado de la carga y clasificacion del catalogo; el anunciante, que gestiona campanas publicitarias en autoservicio; y dos servicios externos, la pasarela de pago y el proveedor de derechos de contenido. Funcionalmente, el sistema debe ofrecer un catalogo navegable con busqueda y recomendaciones personalizadas, reproduccion adaptativa segun el ancho de banda disponible, perfiles multiples por cuenta, historial de visualizacion, facturacion recurrente e insercion de publicidad en los niveles gratuitos.

Las tendencias hacia resoluciones cada vez mas altas y hacia experiencias de baja latencia imponen requisitos estrictos de ancho de banda de extremo a extremo y de retardo maximo tolerable, lo que obliga a que la arquitectura contemple desde el diseno mecanismos de entrega de contenido distribuidos y no unicamente el escalado del servidor de origen[2]. Estos requisitos, sumados a un volumen de 12 TB de video distribuidos en 8 000 titulos y un equipo de desarrollo de solo 12 personas, condicionan de forma directa la eleccion arquitectonica que se justifica en las siguientes secciones.

2. Marco metodologico

El razonamiento arquitectonico de este trabajo se apoya en dos normas internacionales y en el catalogo de patrones consolidado por la disciplina de arquitectura de software. La estructura del razonamiento -identificacion de partes interesadas, preocupaciones y vistas arquitectonicas- sigue el marco de descripcion arquitectonica ISO/IEC/IEEE 42010:2022[3], mientras que la seleccion y priorizacion de atributos de calidad se apoya en el modelo de calidad de producto de software de la norma ISO/IEC 25010:2023[4], que sustituye a la edicion de 2011 e incorpora los atributos de capacidad de interaccion (interaction capability) y de seguridad funcional (safety) como caracteristicas independientes.

El catalogo de patrones arquitectonicos del que se seleccionan las alternativas evaluadas -monolito modular, arquitectura en capas, microservicios, arquitectura orientada a eventos, arquitectura orientada a servicios (SOA) y serverless- proviene de los textos de referencia de Bass, Clements y Kazman[5] y

de Richards y Ford[6], que sistematizan las decisiones de trade-off entre atributos de calidad para cada estilo arquitectónico. Finalmente, la comunicación visual de la arquitectura resultante utiliza la notación C4 propuesta por Brown[7], que distingue niveles de abstracción (contexto, contenedores, componentes y código) y resulta apropiada para un diagrama de Nivel 2 (contenedores) como el que se presenta en la sección 6.

3. Atributos de calidad priorizados

A partir de los datos cuantitativos declarados en el escenario, se identificaron y priorizaron cinco atributos de calidad de la norma ISO/IEC 25010:2023[4]. La priorización privilegia primero la continuidad del servicio y el desempeño percibido por el usuario final, por ser los factores que más directamente determinan la retención de suscriptores en plataformas de streaming[1], [2], y en segundo lugar la capacidad del equipo reducido de mantener el sistema en el tiempo.

Tabla 1. Atributos de calidad priorizados según ISO/IEC 25010:2023

Prioridad	Atributo de calidad (ISO/IEC 25010:2023)	Justificación cuantitativa (tarjeta del escenario)
1	Fiabilidad (disponibilidad)	El SLA exige 99,9 % de disponibilidad mensual, equivalente a un tope de caída de aproximadamente 43 minutos al mes; con 50 000 usuarios concurrentes en hora pico, cualquier interrupción tiene impacto masivo inmediato.
2	Eficiencia de desempeño	La latencia de inicio de reproducción debe mantenerse por debajo de 200 ms en el percentil 95, con un ratio de acierto de cache del CDN de al menos 0,95 (miss/hit $\leq 0,05$).
3	Flexibilidad (escalabilidad)	El sistema debe soportar 50 000 usuarios concurrentes en horario prime time sobre un catálogo de 12 TB y 8 000 títulos, con un equipo de desarrollo acotado a 12 personas.
4	Seguridad	El tratamiento de datos de suscriptores y pagos debe cumplir la LOPDP (2021) y los objetivos de recuperación RPO de 15 minutos y RTO de 60 minutos frente a incidentes.
5	Mantenibilidad	Un equipo de solo 12 personas debe evolucionar de forma continua un catálogo de 8 000 títulos y sus mecanismos de recomendación sin detener el servicio, lo que exige unidades de despliegue pequeñas y acotadas.

3.1. Fiabilidad y eficiencia de desempeño

La eficiencia de desempeño y la fiabilidad se refuerzan mutuamente en este dominio: estudios sobre entrega de video adaptativo muestran que la proximidad geográfica del contenido cacheado en el CDN reduce de forma directa tanto la latencia de arranque como la probabilidad de interrupciones de reproducción (rebuffering) bajo carga alta[1], [2]. Adicionalmente, patrones de tolerancia a fallos como el circuit breaker han demostrado en sistemas distribuidos de alta concurrencia una reducción medible de la propagación de fallas entre servicios y una mejora sostenida de la disponibilidad efectiva por encima del 99,9 % en escenarios de carga pico[8], lo que resulta directamente aplicable al requisito de disponibilidad mensual del 99,9 % declarado en el escenario.

3.2. Flexibilidad y mantenibilidad

El atributo de flexibilidad para escalar horizontalmente el subsistema de entrega de contenido y el de transcodificación se prioriza por encima de otros atributos como la compatibilidad o la capacidad de interacción, que si bien son relevantes, no están condicionados por cifras críticas en el escenario. La mantenibilidad se incorpora como quinto atributo priorizado porque el equipo de desarrollo declarado (12 personas) es reducido frente al alcance funcional del sistema, por lo que unidades de despliegue pequeñas y con responsabilidad única reducen el costo cognitivo de cada cambio.

3.3. Seguridad

El tratamiento de datos personales de los suscriptores y de los medios de pago exige cumplir la Ley Orgánica de Protección de Datos Personales del Ecuador (LOPD, 2021), lo que en términos arquitectónicos se traduce en requisitos equivalentes a los que impone el Reglamento General de Protección de Datos europeo (GDPR) sobre trazabilidad, consentimiento y control de acceso a los datos, y que pueden implementarse mediante políticas de acceso adjuntas a los propios datos (sticky policies) y mediante segregación de los servicios que procesan información sensible[9]. Este atributo se desarrolla con mayor detalle en la sección 5.3.

4. Selección del patrón arquitectónico

Se selecciona una arquitectura de microservicios como patrón principal, combinada con una arquitectura orientada a eventos (EDA) para la comunicación asíncrona entre los servicios de transcodificación, catálogo, recomendaciones y facturación. Esta combinación es consistente con el catálogo de patrones empresariales descrito por Bass, Clements y Kazman[5] y por Richards y Ford[6], y responde a un patrón de diseño recurrente en plataformas de streaming a gran escala, donde cada capacidad de negocio (catálogo, transcodificación, suscripción, publicidad) se despliega y escala de forma independiente, siguiendo topologías ya documentadas para el procesamiento de video en tiempo real[10].

La comunicación asíncrona mediante un bus de eventos se incorpora específicamente para desacoplar el flujo de trabajo de transcodificación, que es de larga duración, del resto del sistema. Evidencia empírica reciente sobre arquitecturas orientadas a eventos en sistemas de microservicios reporta mejoras medibles en tiempo de respuesta y en tasa de error frente a comunicación síncrona basada en API, a costa de un mayor consumo de recursos de cómputo[11], lo cual es un trade-off aceptable dado que el cuello de botella real del escenario es la latencia de reproducción, no el uso de CPU del bus de mensajería.

Tabla 2. Matriz de decisión: patrones candidatos frente a atributos priorizados

Patrón candidato	Fiabilidad	Desempeño	Flexibilidad	Mantenibilidad
Monolito modular	Medio	Medio	Bajo	Medio
Arquitectura en capas clásica	Medio	Medio	Bajo	Medio
SOA con bus de servicios empresarial (ESB)	Medio	Bajo	Medio	Bajo
Microservicios + EDA (elegido)	Alto	Alto	Alto	Alto

4.1. Justificación de la elección

Atributo que resuelve: la flexibilidad para escalar de forma independiente el servicio de transcodificación, que es intensivo en CPU, frente al servicio de catálogo, que es intensivo en lectura, sin sobreaprovisionar todo el sistema por igual[10].

Alternativa descartada: un monolito modular fue la primera opción evaluada por su menor complejidad operativa para un equipo de 12 personas, pero fue descartada porque no permite escalar el módulo de transcodificación sin escalar también componentes que no lo requieren.

Trade-off aceptado: se acepta mayor complejidad operativa (orquestración de contenedores, observabilidad distribuida) a cambio de escalabilidad selectiva y de aislamiento de fallos, de modo que una degradación en el servicio de recomendaciones no comprometa la reproducción de video[10], [11].

Riesgos abiertos: la consistencia eventual entre servicios (por ejemplo, entre el registro de un nuevo pago y la habilitación de acceso al catálogo) y el costo de operación de un equipo reducido frente a la complejidad propia de sistemas distribuidos[11].

5. Estrategias transversales de la arquitectura

5.1. Escalabilidad y autoescalado

Cada servicio se despliega en contenedores independientes con políticas de autoescalado propias, en lugar de un único mecanismo de escalado uniforme para toda la aplicación. Trabajos recientes sobre autoescalado de microservicios muestran que el escalado diferenciado por servicio, en función de las características de carga particulares de cada uno, reduce las violaciones de los objetivos de nivel de servicio (SLO) frente al escalado reactivo uniforme basado únicamente en el uso de CPU[12]. Esto es especialmente relevante para el servicio de transcodificación, cuyo patrón de consumo de recursos es muy distinto al del servicio de catálogo o al de recomendaciones.

5.2. Resiliencia y tolerancia a fallos

Se incorporan los patrones de circuit breaker y de reintentos con backoff exponencial en las llamadas sincrónicas del API Gateway hacia la pasarela de pago y el proveedor de derechos, de modo que una falla en un sistema externo no se propague hacia el resto de la plataforma. La evidencia disponible sobre sistemas distribuidos de alta concurrencia reporta reducciones de hasta el 60 % en la propagación de fallas entre servicios al introducir el patrón circuit breaker, junto con mejoras de disponibilidad y de capacidad de atender más usuarios concurrentes durante los picos de tráfico[8].

5.3. Seguridad y cumplimiento normativo

El servicio de suscripción y pago se aísla del resto de la plataforma como límite de confianza independiente, con cifrado de datos en tránsito y en reposo, y con un registro de auditoría de todo acceso a datos personales. Este diseño es consistente con arquitecturas propuestas en la literatura para el cumplimiento de regulaciones de protección de datos, en las que las políticas de acceso viajan junto con el dato (sticky policies) y permiten validar de forma automática el propósito de cada acceso y mantener trazabilidad para auditorías externas[9], un enfoque directamente aplicable al cumplimiento de la LOPDP en el contexto ecuatoriano.

6. Diagrama de arquitectura

El siguiente diagrama, en notación equivalente a C4 Nivel 2 (Contenedores)[7], muestra los actores externos, el CDN de entrega de video, la puerta de enlace de API, los contenedores de servicio y el bus

de eventos que los conecta de forma asincrona. Cada contenedor se etiqueta con su tecnologia de referencia y cada relacion sincrona declara el protocolo utilizado.

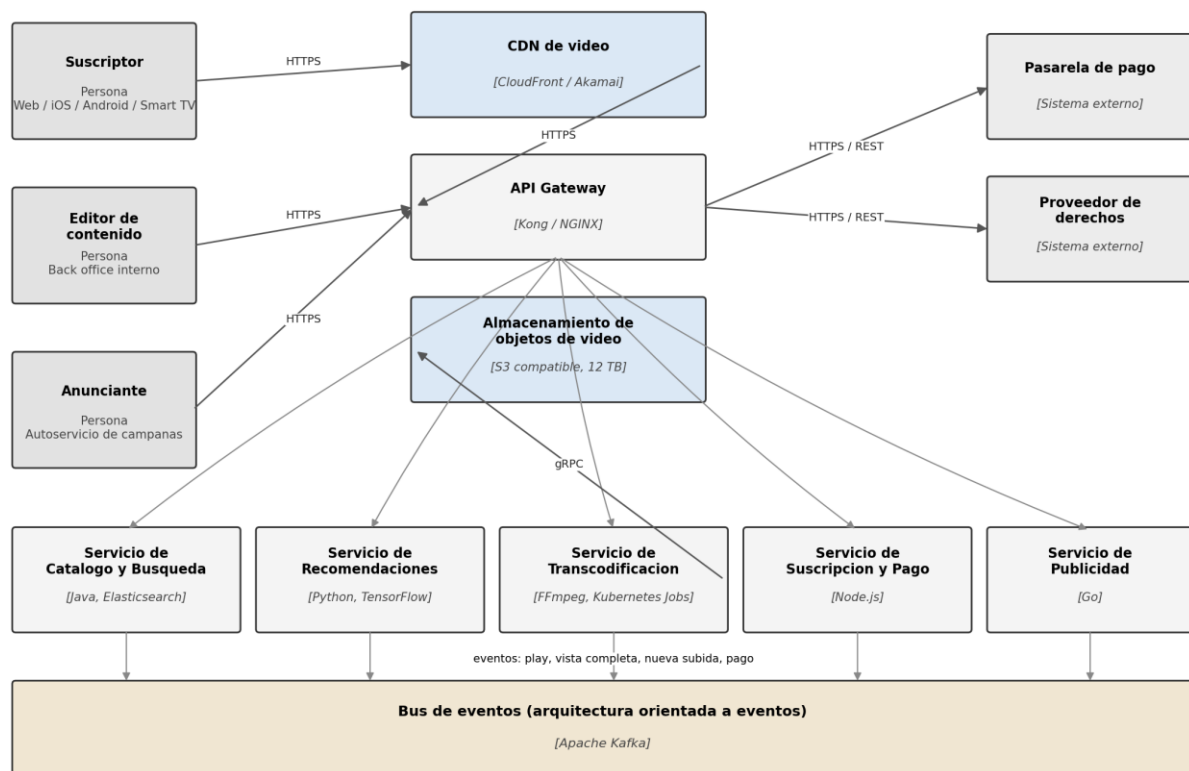


Figura 1. Diagrama de contenedores (C4 Nivel 2) de la plataforma de streaming

El CDN absorbe la mayor parte del trafico de lectura de video, de modo que el API Gateway y los servicios internos solo atienden solicitudes de catalogo, autenticacion, recomendaciones y transacciones. El bus de eventos desacopla la finalizacion de una transcodificacion, la confirmacion de un pago o la senal de visualizacion completa del resto de los servicios, que reaccionan a esos eventos sin conocerse entre si directamente. Esta separacion entre el plano de entrega de contenido (CDN, almacenamiento de objetos) y el plano de control (API Gateway, servicios de negocio) es la que permite escalar el primero de forma independiente del segundo.

7. Analisis de trade-offs y riesgos residuales

La tabla siguiente resume las alternativas descartadas frente al patron elegido, ampliando la comparacion inicial de la seccion 4 con una cuarta alternativa (SOA con bus de servicios empresarial), que es habitual en sistemas corporativos de integracion pero que introduce un punto unico de fallo poco adecuado para el perfil de carga del escenario.

Tabla 3. Alternativas descartadas y trade-offs frente al patron elegido

Alternativa evaluada	Motivo de descarte	Trade-off frente a la opcion elegida
Monolito modular	Un unico proceso desplegable no permite escalar de forma independiente el modulo de transcodificacion, que consume CPU	Se gana simplicidad operativa, pero se pierde la capacidad de escalar cada componente segun su propia carga.

	de forma muy distinta al resto del sistema.	
Arquitectura en capas clasica	Acopla el ciclo de despliegue de todos los modulos y no aisla el trafico de video del trafico transaccional (pagos, perfiles).	Se gana curva de aprendizaje mas baja para un equipo de 12 personas, pero se sacrifica aislamiento de fallos.
SOA con ESB centralizado	El bus centralizado se convierte en un cuello de botella y en un punto unico de fallo cuando el volumen de eventos crece con el catalogo y la concurrencia.	Se gana gobierno centralizado de integraciones, pero se pierde la escalabilidad horizontal independiente de cada servicio.
Serverless / FaaS puro	La latencia de arranque en frio y el limite de tiempo de ejecucion penalizan procesos de transcodificacion largos y flujos de datos de alto volumen y baja latencia.	Se gana coste marginal bajo en trafico esporadico, pero se pierde previsibilidad de latencia p95 bajo carga sostenida.

Como riesgo residual transversal, el equipo asume que la consistencia eventual entre el servicio de suscripcion y el servicio de catalogo puede generar una ventana breve en la que un pago confirmado aun no habilite el acceso al contenido; este riesgo se mitiga con una notificacion inmediata al usuario y con un mecanismo de reintento automatico del lado del cliente, en lugar de forzar una transaccion distribuida sincrona que degradaria la disponibilidad general del sistema.

8. Registro breve de la decision arquitectonica

Siguiendo la disciplina de decisiones arquitectonicas versionadas propuesta por Nygard[13], se resume la decision en cuatro puntos:

Contexto: la necesidad de escalar de forma independiente un subsistema de transcodificacion intensivo en CPU frente a un catalogo intensivo en lectura, con un equipo de desarrollo reducido.

Decision: microservicios con comunicacion orientada a eventos (EDA).

Consecuencia positiva: se refuerza la flexibilidad de escalado y el aislamiento de fallos entre servicios.

Consecuencia negativa: se acepta consistencia eventual entre servicios y un mayor esfuerzo de observabilidad distribuida para un equipo pequeno.

9. Guion de exposicion individual

Dado que el taller se desarrolla de forma individual, el guion de 10 minutos originalmente distribuido entre los integrantes de un equipo se adapta a una sola persona, conservando los mismos bloques de contenido y tiempos aproximados sugeridos en la guia del taller:

Minuto 0-1, contexto del escenario: presentar el modelo de negocio de la plataforma, los actores principales y las cifras clave (50 000 usuarios concurrentes, 12 TB de catalogo, SLA de 99,9 %).

Minuto 1-3, atributos de calidad priorizados: recorrer la Tabla 1 explicando por que fiabilidad y eficiencia de desempeno dominan sobre los demas atributos, con las cifras que lo sustentan.

Minuto 3-5, patron elegido y justificacion: presentar la Tabla 2 y explicar por que microservicios con EDA superan a las tres alternativas descartadas en los cuatro atributos priorizados.

Minuto 5-8, recorrido del diagrama: explicar la Figura 1 contenedor por contenedor, distinguiendo el plano de entrega de contenido del plano de control y el rol del bus de eventos.

Minuto 8-10, trade-offs y riesgos abiertos: cerrar con la Tabla 3 y el riesgo residual de consistencia eventual descrito en la seccion 7, dejando espacio para las preguntas cruzadas del grupo.

10. Conclusion

El analisis cuantitativo de los atributos de calidad del escenario -disponibilidad de 99,9 %, latencia p95 menor a 200 ms y 50 000 usuarios concurrentes en hora pico- respalda una arquitectura de microservicios apoyada en un CDN para la entrega de video y en un bus de eventos para desacoplar los procesos de mayor duracion. Las estrategias transversales de autoescalado diferenciado, tolerancia a fallos mediante circuit breaker y segregacion del dominio de pagos abordan de forma especifica cada uno de los cinco atributos priorizados en la Tabla 1. El principal riesgo que el equipo de desarrollo debe gestionar activamente es la complejidad operativa adicional que este patron introduce frente a alternativas mas simples, dado el tamano acotado del equipo declarado en el escenario.

Referencias

- [1] Y. Zhang and R. Boutaba, "Cloud-Based Video Streaming Services: A Survey," arXiv:2011.14976, Nov. 2020. [Online]. Available: <https://arxiv.org/pdf/2011.14976>
- [2] R. Farahani, "Network-Assisted Delivery of Adaptive Video Streaming Services through CDN, SDN, and MEC," arXiv:2403.16951, Mar. 2024. [Online]. Available: <https://arxiv.org/abs/2403.16951>
- [3] ISO/IEC/IEEE, "ISO/IEC/IEEE 42010:2022 - Software, systems and enterprise - Architecture description," International Organization for Standardization, Geneva, Switzerland, Nov. 2022.
- [4] ISO/IEC, "ISO/IEC 25010:2023 - Systems and software engineering - Systems and software quality requirements and evaluation (SQuaRE) - Product quality model," International Organization for Standardization, Geneva, Switzerland, Nov. 2023.
- [5] L. Bass, P. Clements, and R. Kazman, Software Architecture in Practice, 4th ed. Boston, MA, USA: Addison-Wesley Professional, 2021.
- [6] M. Richards and N. Ford, Fundamentals of Software Architecture: An Engineering Approach. Sebastopol, CA, USA: O'Reilly Media, 2020.
- [7] S. Brown, The C4 Model for Visualising Software Architecture. Leanpub, 2023.
- [8] B. Barua and M. S. Kaiser, "Enhancing Resilience and Scalability in Travel Booking Systems: A Microservices Approach to Fault Tolerance, Load Balancing, and Service Discovery," arXiv:2410.19701, Oct. 2024. [Online]. Available: <https://arxiv.org/pdf/2410.19701>
- [9] "Towards a GDPR-compliant cloud architecture with data privacy controlled through sticky policies," PMC, National Center for Biotechnology Information, 2024. [Online]. Available: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC11041943/>
- [10] "A Scalable Microservices Architecture for Real-Time Data Processing," International Journal of Advanced Computer Science and Applications, vol. 16, no. 9, 2025. [Online]. Available: https://thesai.org/Downloads/Volume16No9/Paper_5-A_Scalable_Microservices_Architecture.pdf
- [11] J. Arafat et al., "Next-Generation Event-Driven Architectures: Performance, Scalability, and Intelligent Orchestration Across Messaging Frameworks," arXiv:2510.04404, Oct. 2025. [Online]. Available: <https://arxiv.org/abs/2510.04404>

[12] M. Dashtbani and L. Tahvildari, "STaleX: A Spatiotemporal-Aware Adaptive Auto-scaling Framework for Microservices," arXiv:2501.18734, Jan. 2025. [Online]. Available: <https://arxiv.org/abs/2501.18734>

[13] M. Nygard, "Documenting architecture decisions," Cognitect Blog, Nov. 2011.